do this? It loads the source of your CoffeeScript program, compiles it, and then tells the browser to evaluate the compiled JavaScript.

That sounds too good to be true! Unfortunately it is. Sending CoffeeScript directly to the browser comes with a performance disadvantage because every time the browser loads your program, it has to first load the CoffeeScript, then compile the program, and *then* run it. Regardless, you don't want to compile your CoffeeScript manually. You need a third alternative.

11.1.3. Automatic compilation

Manual compilation is not only annoying; it can hurt your interface. To develop good user interfaces, it helps to have frequent and direct feedback from the working interface. If you have to manually compile every time, you'll lose your flow.

To keep the immediacy when writing CoffeeScript programs for web browsers, use the `-w` (watch) compiler argument to automatically recompile every time you save a change to a file. Invoke the compiler with the watch option and the filenames you want to compile:

```
> coffee -c -w status.coffee
```

Each time you save a change, you're notified that the compiler has run:

```
# 21:52:16 - compiled status.coffee
# 21:52:57 - compiled status.coffee
# 21:53:00 - compiled status.coffee
```

Now that you're happily compiling away (or not), it's time to get back to the task of building Agtron's dashboard.

11.2. Communicating with the outside world

Before you can display any data about Agtron's servers, you need to get that data from somewhere. Suppose Agtron has already implemented a data